1 Introduction and Preliminaries

1.1 Introduction

This laboratory work focused on applying embedded real-time control principles using the Turtlbot mobile robot. Across four labs, students integrated sensors, actuators, and control algorithms in real-world scenarios. In Lab 1, I^2C communication was used to interface a reflectance sensor array and a keypad with the STM32 microcontroller. Lab 2 involved using IMU measurements to control servo motors for stabilizing a camera, demonstrating closed-loop control based on orientation feedback. In Lab 3, motor control was implemented using encoder feedback to estimate rotation and regulate speed. Finally, Lab 4 combined sensor data processing and control by developing a line-following algorithm based on surface reflectivity. Together, these labs provided hands-on experience in embedded system integration and real-time control implementation.

1.2 Hardware Overview

The robot used in this laboratory activity is the TurtleBot, a compact differential-drive mobile robot designed for embedded control and sensor integration experiments. It is equipped with various hardware components that support real-time control, perception, and interaction with the environment. The main components of the Turtlbot used in the lab are:

- **STM32 Microcontroller:** Acts as the main processing unit, handling sensor readings, control algorithms, and actuator commands. It provides real-time performance and efficient peripheral management.
- **Two DC Motors with Integrated Encoders:** These motors drive the robot's wheels and allow precise measurement of wheel rotation through built-in position encoders, enabling accurate odometry and velocity control.
- **Inertial Measurement Unit (IMU):** Provides data on angular velocity and linear acceleration, essential for orientation estimation and motion analysis.
- **Servo Motors for Camera Pan and Tilt:** Two servo motors allow control of the camera's orientation. The servo motors are also used to stabilize the camera.
- **Pololu QTR Reflectance Sensor Array:** Used for line detection, this sensor array enables the Turtlbot to follow lines by sensing changes in surface reflectivity.
- **Keypad:** Provides a user interface for manual input or control commands.
- **LED Indicators:** Used for status indication.

1.3 Software Overview

The development and testing of the Turtlbot's control and sensing capabilities were supported by a set of software tools tailored for embedded system development and data analysis. The following tools were used throughout the laboratory activities:

- **STM32CubeIDE:** An integrated development environment used for writing, compiling, and debugging C code on the STM32 microcontroller. It provides a unified workspace for managing projects, accessing peripheral registers, and performing real-time debugging.
- **STM32CubeMX:** A graphical configuration tool used to initialize and configure the microcontroller's peripherals, clock settings, GPIOs, and middleware. CubeMX simplifies the setup of timers, ADCs, communication interfaces (e.g., I^2C, UART), and generates initialization code compatible with STM32CubeIDE.
- **MATLAB:** Used for data acquisition, visualization, and post-processing of experimental results. MATLAB facilitated the analysis of sensor readings, control responses, and system behavior, allowing for validation and tuning of the implemented algorithms.

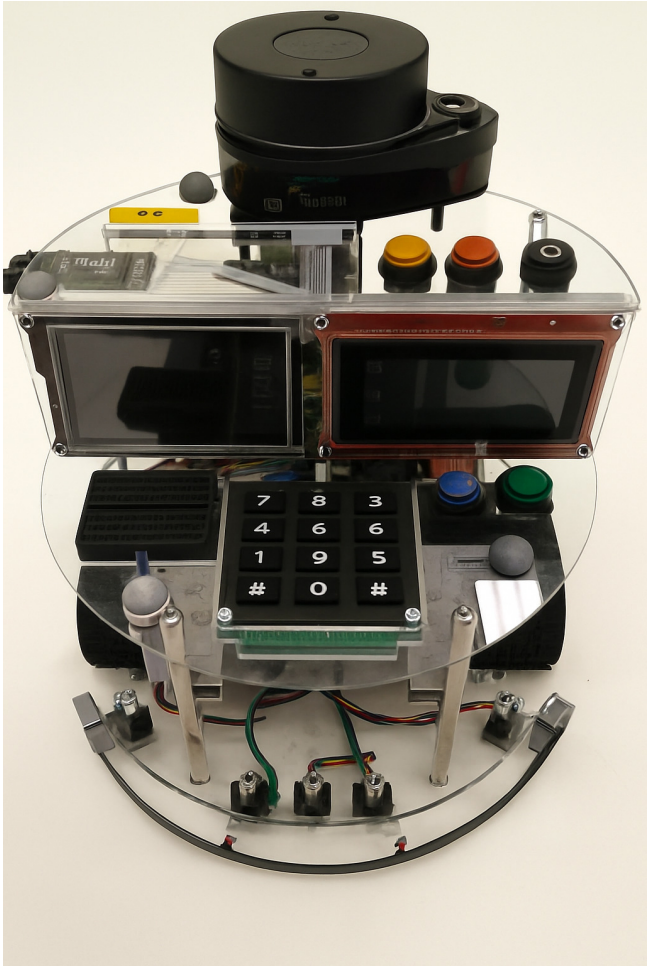


Figure 1: Turtle Bot

4 Lab 3: Motor Control

4.1 Introduction

The objective of this lab activity is to implement a PI controller to control the angular speed of the robot wheel, rotating under the action of a DC motor. The problem involves interfacing with the sensor (encoder) and actuator (motor) and implementing the control logic in a closed-loop environment.

4.2 Problem Analysis

The problem consists of three main parts:

1. **Interfacing with the encoder:** The encoder generates a pulse when the position changes, so the objective is to compute the current angular speed of the wheel using the encoder measurements.
2. **Calculating the control signal:** Implementing the logic for the PI controller that takes the error of the angular speed and gives a control signal to be executed by the motor.
3. **Interfacing with the motor driver:** This means converting the control signal to a proper PWM signal that can be fed to the motor driver which is responsible of driving the motor.

A schematic for this feedback controller is shown in Figure(8).

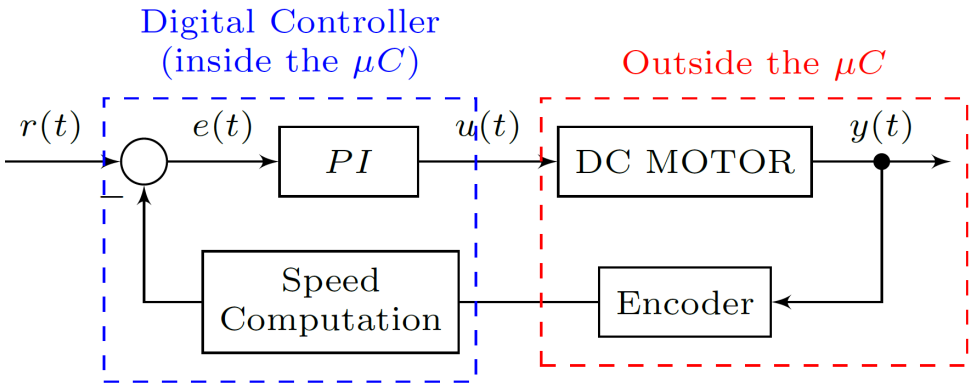


Figure 8: Feedback Control Scheme.

4.3 Theoretical Background

4.3.1 Motor Model

A DC motor consists of two main components: the stator and the rotor. The stator is the stationary part of the motor and is responsible for generating a magnetic field, either through permanent magnets or electromagnetic windings. The rotor, which is the rotating part, contains windings that produce a magnetic field when an electric current flows through them. A commutator, typically composed of brushes in contact with the rotor windings, acts as a mechanical switch that ensures continuous rotation by periodically reversing the current direction. The interaction between the magnetic field of the stator and that of the rotor generates a repelling force, producing a torque that causes the rotor to spin. This torque is directly proportional to the current through the rotor winding, described by the equation $\tau = k_t i$ where k_t is the torque constant.

The equations of the motor consists of two parts:

- Electrical: From Kirchhoff's Voltage Law (KVL) to the armature circuit

$$\begin{aligned} V_a(t) &= R_a i_a(t) + L_a \frac{di_a(t)}{dt} + V_b(t) \\ &= R_a i_a(t) + L_a \frac{di_a(t)}{dt} + K_b \omega(t) \end{aligned} \tag{6}$$

- Mechanical: From Newton's second law to the rotor

$$J \frac{d\omega(t)}{dt} = T_m(t) - b\omega(t) = K_t i_a(t) - b\omega(t) \tag{7}$$

Using the Laplace transform on (6) and (7) and assuming $K = K_t = K_b$

$$G(s) = \frac{\Omega(s)}{V_a(s)} = \frac{K}{L_a J s^2 + (R_a J + L_a b)s + (R_a b + K^2)} \tag{8}$$

Symbol	Description	Units
$V_a(t)$	Armature voltage (Input voltage)	V
$i_a(t)$	Armature current	A
R_a	Armature resistance	Ω
L_a	Armature inductance	H
$V_b(t)$	Back EMF (Electromotive Force)	V
K_b	Back EMF constant	V/(rad/s)
$\omega(t)$	Angular velocity of the rotor	rad/s
$T_m(t)$	Motor torque developed by the motor	Nm
K_t	Torque constant	Nm/A
J	Moment of inertia of rotor and load	kg·m^2
b	Viscous friction coefficient (damping)	Nm/(rad/s)

Table 1: DC Motor Variables and Parameters

4.3.2 Motor Driver

The motor driver is a chip that can convert the PWM input into a proper output, both in terms of voltage and current, to drive the motor. The chosen chip is DRV8871 by Texas Instruments. is a brushed DC motor driver that can control the motor bidirectionally by implementing a H-bridge. Reported in Figure (9) the pinout of the motor driver DRV8871 and the pins discription in table (2).

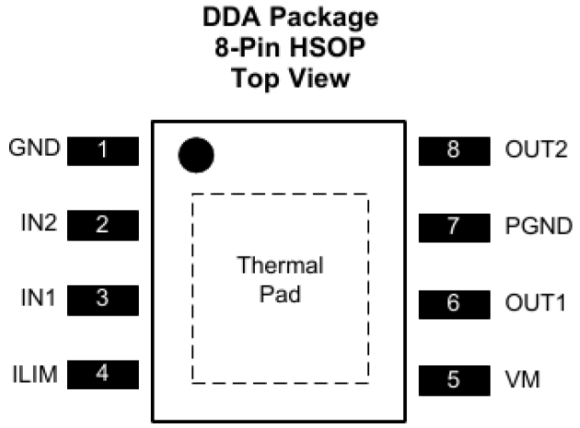


Figure 9: DRV8871 pinout

Pin name	Pin number	Pin type	Description
IN1	3	Input	Input of the chip that allows controlling the output of the H-bridge. These pins receive the PWM signal from the microcontroller.
IN2	2	Input	-
OUT1	6	Output	-
OUT2	8	Output	H-bridge output that goes to the motor.

Table 2: Pin configuration table for the H-bridge

Bridge control

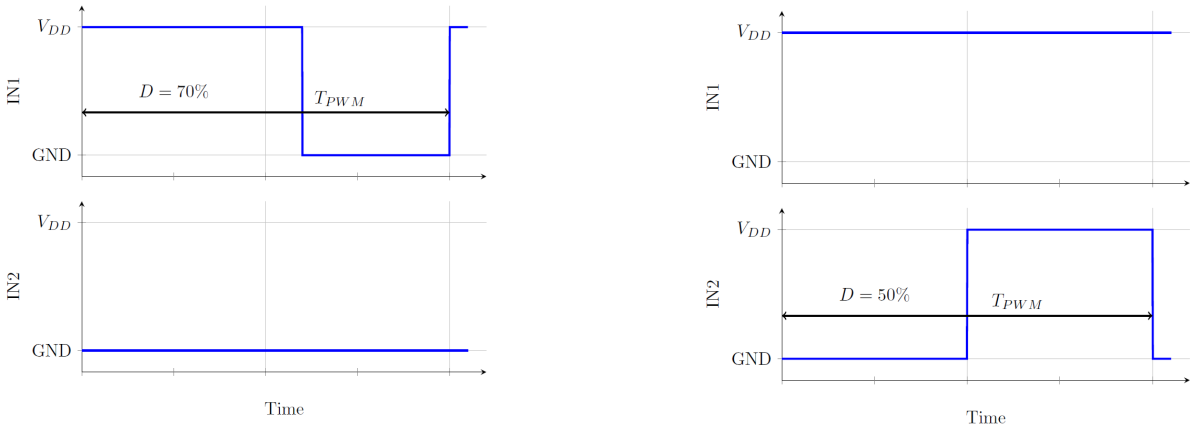
The H-bridge is an electrical circuit used to select the desired driving mode for the motor based on the input of the driver pins. In Table (3), the four configuration of the input pins are reported with the corresponding

IN1	IN2	OUT1	OUT2	Mode name
0	0	Hi-z	Hi-z	*Coast*
0	1	L	H	*Reverse*
1	0	H	L	*Forward*
1	1	L	L	*Brake*

Table 3: Motor driving modes.

mode. The coast mode allows the motor to coast to a stop, which means that the motor is not driven and will move by inertia. On the other hand, the brake mode will stop the motor faster.

When the motor is driven using PWM signal with duty cycle less than 100%, it's possible to alternate between forward and coast modes or forward and brake mode as shown in Figure (10).



(a) Forwrad-Coast mode, speed $\propto$ D

(b) Forward-Break mode, speed $\propto$ 1-D

Figure 10: PWM signal for different driving modes.

4.3.3 Position Encoder

The position encoder is a sensor that converts position into an electrical signal. In this context, we focus on a rotary encoder, which is specifically designed to measure angular or rotational positions. More precisely, we use an incremental encoder, which produces a series of pulses corresponding to changes in position, allowing the detection of relative movement rather than absolute position.

The used encoder is of quadrature type which means that the encoder employs two outputs A and B, which are called quadrature outputs, as they are 90 degrees out of phase. The direction of the motor depends on which phase's signal leads over the other. A counter can be used to measure the number of pulses (a pulse can be counted when an edge happens) the number of pulses per round depends on the counting mode. Figure (11) illustrates the different counting modes.The corresponding pulse per round (PPP) is reported in Table (4).

Mode	ppr motor-side	ppr wheel-side
1X	8	960
2X	16	1920
4X	32	3840

Table 4: Pulses per round for different counting modes with gearbox of ratio of 120:1.

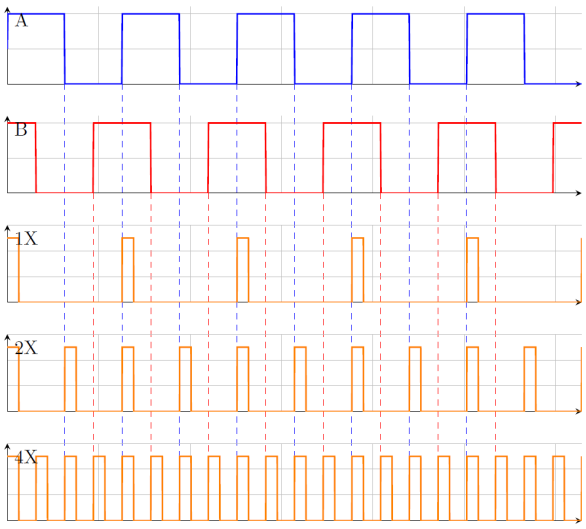


Figure 11: Counting modes illustration.

For example, if the 4X mode is used, the encoder provides 32 pulses for one rotation of the motor which means that for the wheel it's $32 * 120 = 3840$ pulses for one rotation.

4.4 Implementation and Code Structure

4.4.1 Hardware Setup and Peripheral List

- TIM3 (16bit, general-purpose timer), channel 1 and channel 2: Encoder for motor 1.
- TIM4 (16bit, general-purpose timer), channel 1 and channel 2: Encoder for motor 2.
- TIM6 (16bit, basic timer): Provides the clock for the controller.
- TIM8 (16bit, advanced timer), channel 1 and channel 2: PWM for motor 1.
- TIM8 (16bit, advanced timer), channel 3 and channel 4: PWM for motor 2.

The Figure (12) shows the scheme for hardware connectinos.

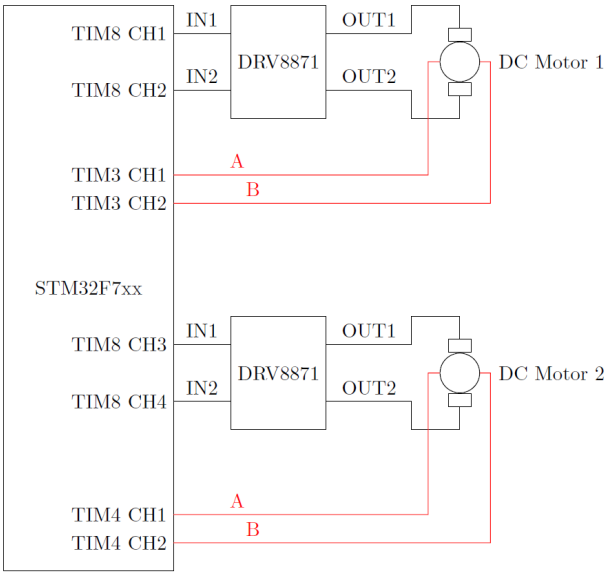


Figure 12: Connection scheme for the timers.

4.4.2 Calculating speed from the encoder measurements

The two main focus points in this section is

- Because the timer counter period is set equal to the number of pulses per round, 3840 in case of 4X counting mode, there might be an underflow or overflow in the reading. This is checked and corrected before computing the speed.
- Computing the speed from the encoder measurements.

The logic behind the solution of the first point is to check first whether the timer is counting up or down. For example, if the timer is counting up, overflow happens when the current reading is less than the previous one. To correct the reading in this case, the current reading is added to the interval between the previous reading and the the maximum reading. Similar logic is implemented for the case of underflow. The full implementation is given in code (12).

Listing 12: Getting encoder measurement and correcting over/under flow

```
# define TIM3_ARR_VALUE 3840 // assuming we are using 4X encoder mode
float ReadCurrentSpeed(int motorNumber){
    uint32_t CurrentCount = 0;
    int32_t DiffCount = 0;
    static uint32_t PreviousCount = 0;
    // get diff count for specified motor htim3 for motor 1 and htim4 for motor 2
    switch (motorNumber){
        case 1:
            CurrentCount = __HAL_TIM_GET_COUNTER(&htim3);
            // calculate diff count of motor 1
            if (__HAL_TIM_IS_TIM_COUNTING_DOWN(&htim3))
             {
                /* check for counter underflow */
                if (CurrentCount <= PreviousCount) // Case of no underflow
                    DiffCount = CurrentCount- PreviousCount;
                else // Case of underflow
                    DiffCount =-((TIM3_ARR_VALUE+1)- CurrentCount)- PreviousCount;
             }
             else
             {
                /* check for counter overflow */
                if (CurrentCount >= PreviousCount) // Case of no overflow
                    DiffCount = CurrentCount- PreviousCount;
                else    // Case of overflow
                    DiffCount = ((TIM3_ARR_VALUE+1)- PreviousCount) + CurrentCount;
             }
            PreviousCount = CurrentCount; // Storing the current value
        .
        .
        .
    return currentSpeed;
}
```

As for the second point, the following formula is implemented in code (13)

$$
\begin{aligned}
no.revolutions &= \frac{count\ difference}{PPP(3840)} \\
\omega_{wheel} &= \frac{\text{no. revolutions}}{T_s} * 60\ [rpm]
\end{aligned}
\tag{9}
$$

Listing 13: Wheel speed calculation

```
# define TIM3_ARR_VALUE 3840 // assuming we are using 4X encoder mode
float ReadCurrentSpeed(int motorNumber){
        uint32_t CurrentCount = 0;
        int32_t DiffCount = 0;
        static uint32_t PreviousCount = 0;
        // get diff count for specified motor htim3 for motor 1 and htim4 for motor 2
    .
```

```
8       .
9       .
10          float currentSpeed = 0;
11          // Current speed in rpm
12          currentSpeed = ((float)DiffCount /(float)TIM3_ARR_VALUE)/ TS * 60.0;
13
14      return currentSpeed;
15  }
```

4.4.3 Control Logic

To get an initial value for the controller gains, the *MATLAB* toolbox *PID Tuner* is used with the approximated motor transfer function given by

$$P(s) = \frac{\mu}{1+Ts} \cdot e^{-\tau s} \frac{\text{rad/s}}{V} \tag{10}$$

where $\mu = 3.552, \quad T = 0.217, \quad \tau = 0.2$

The implemented controller is a PI controller. The noticeable point in the controller is keeping memory of the value of the integrator which is done by declaring a *static* variable *Iterm* to store the value of the integrator. An Anti-windup is implemented to solve the integrator winding phenomena. The anti-windup, basically, stops the integrating if the output is saturated and the controller output still grows in the wrong direction. see code (14) for the full implementation.

Listing 14: PI Controller with Anti windup

```
1  #define P_gain 0.8
2  #define I_gain 7.5563
3  float calculateControlSignal(float currentSpeed, float referenceSpeed, float* I_term){
4       // PI Controller
5       float error = (referenceSpeed - currentSpeed)*RPM2RADS; //rad/sec
6       float P_term = P_gain*error;
7       float Output_temp = 0;
8
9       *I_term = *I_term + I_gain*TS*error;
10      Output_temp = P_term+*I_term; // volt
11      //float Output_temp = P_term+*I_term; // volt
12      float Output_clamped = clamp(Output_temp);
13
14    // Anti wind-up
15      bool is_saturated = (Output_clamped != Output_temp);
16      bool worsens_saturation = (error * Output_temp > 0);
17      if((is_saturated)&&(worsens_saturation)){
18          *I_term = *I_term - I_gain*TS*error;
19      }
20      return Output_clamped;
21 }
```

4.4.4 Motor Control Routine

The ideal sequence of operation is the following:

1. Wait for clock interrupt: the interrupt is raised, in our case by the timer TIM6
2. Read the encoder value and compute the angular speed
3. Compute the error with respect to the reference signal and compute the control signal

Notice that the time required to measure, convert, compute the control signal and provide the proper output from the controller, needs to be less than the sampling period. The control algorithm can be implemented directly inside the timer callback. The motor control routine is shown in code (15).

Listing 15: Motor Control Routine

```
1  /* Speed ctrl routine */
```

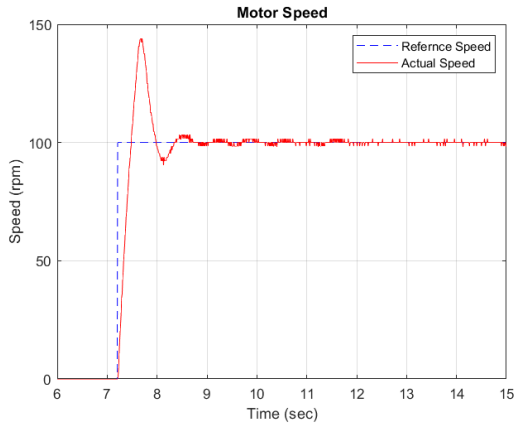
```
2  if(htim->Instance == TIM6)
3  {
4      // Read the speed from the motors
5      float currentSpeed1 = ReadCurrentSpeed(1);
6      float referenceSignal1 = 0;
7      uint32_t ms = HAL_GetTick();
8      float time = ms / 1000.0f;
9
10     if (time < 10.0f){
11         referenceSignal1 = 0.0f;
12     }else if(time > 10.0f){
13         referenceSignal1 = 160.0f;
14     }
15
16     float Output_1 = calculateControlSignal(currentSpeed1, referenceSignal1, &I_term_1);
17     // Set the correct PWM Value
18     int32_t duty1;
19     //calculate duty properly
20     duty1 = Output_1*V2DUTY; //Linear relation between the duty and the voltage
21
22     ForwardBreak(1, duty1);
23         //ForwardCoast(1, duty1);
24 }
```

The functions *ForwardBreak* and *ForwardCoast* is an implementation of the driving modes illustrated in Figure (10). The code is given in Appendix (A).

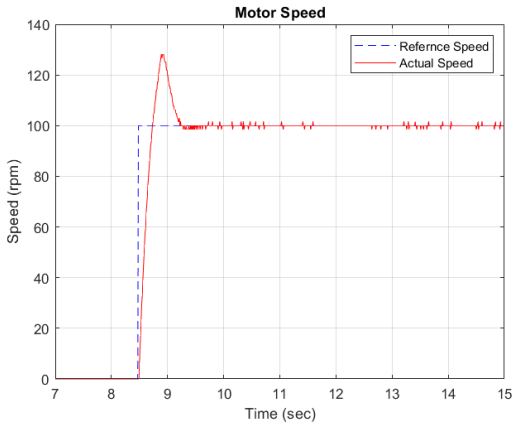
4.5 Results

4.5.1 Controller Step Response

The controller has been tested with a constant speed reference to evaluate its performance and to compare between using the *Forward Coast* and *Forward Break* modes. The result in Figure (13) shows that the *Forward Break* mode is more precise while the response in *Forward Coast* mode has more oscillations and larger overshoot. For this fact, all the tests will be done using the *Forward Break* mode.



(a) Forward Coast mode



(b) Forward Break mode

Figure 13: Comparison between forward break and forward coast driving mode.

4.5.2 Anti-windup Performance

To test the anti-windup, the reference signal is set to exceed the maximum that the motor can output. As shown in Figures (14, 15, 16), the Anti-windup prevents the integrator winding by holding the integrator value when the output saturates.

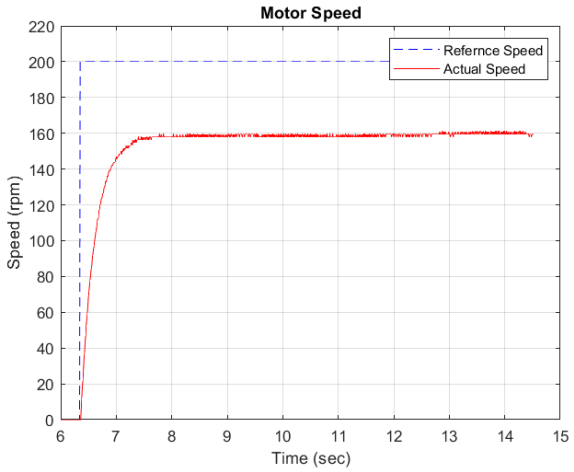
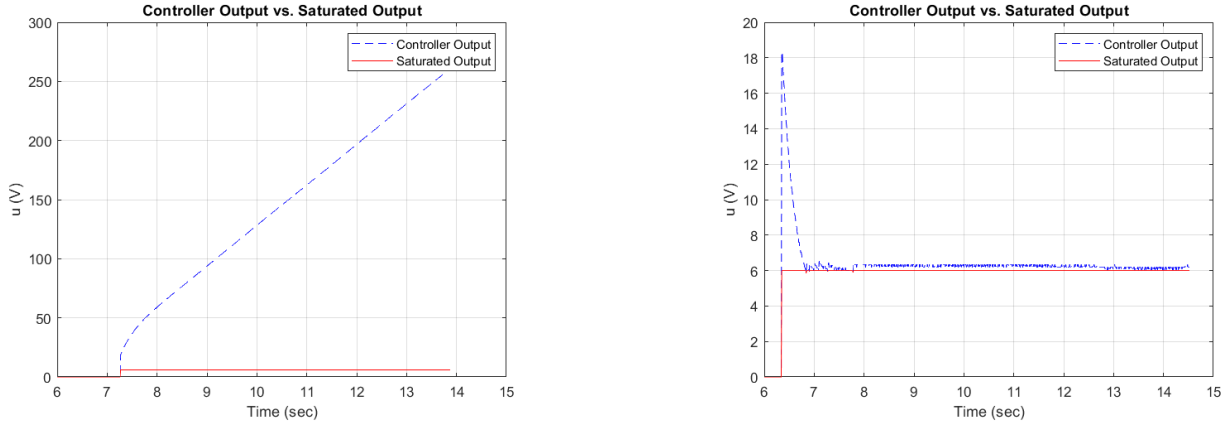


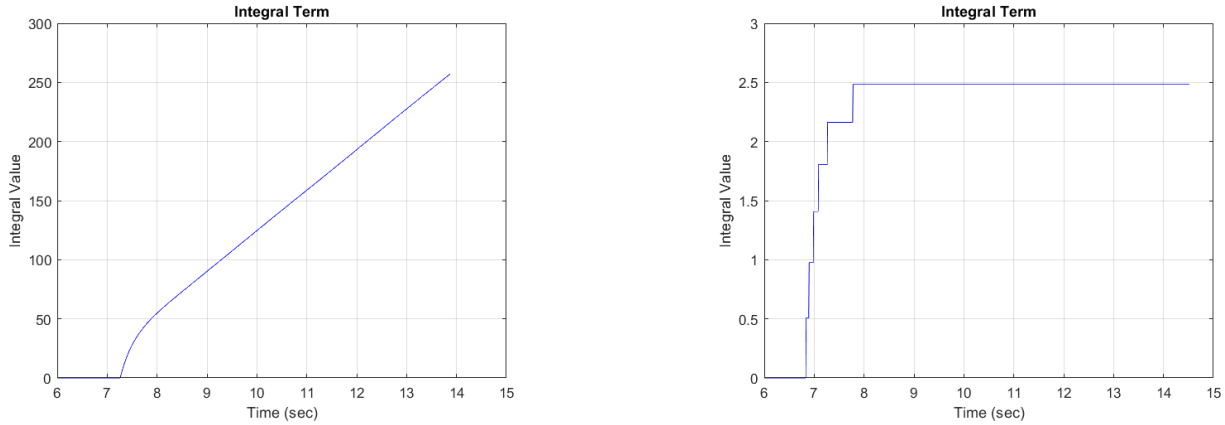
Figure 14: Motor speed in saturation.



(a) No Anti-windup

(b) Using Anti-windup

Figure 15: Effect of using Anti-windup on the controller output.

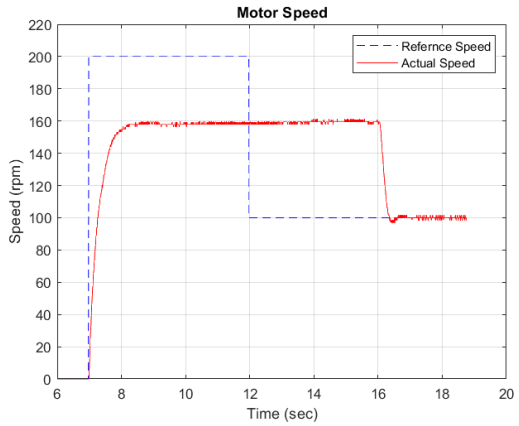


(a) No Anti-windup

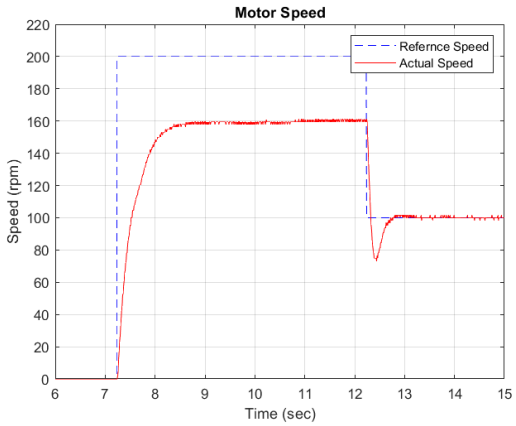
(b) Using Anti-windup

Figure 16: Effect of using Anti-windup on the Integral term.

Another test is done to showcase the effectiveness of using Anti-windup to recover from saturation. The result in Figure (17) shown that without using Anti-windup the motor is still saturated after reducing the reference speed to a smaller value below the motor limit. On the other hand, the Anti-windup is able to recover fast.



(a) No Anti-windup



(b) Using Anti-windup

Figure 17: Effect of using Anti-windup to recover from saturation.